Database Development and Design (SOE201)

Lecture 2b: B^+-Tree Indexing

Dr. Wei Wang

AIAT

Learning Outcomes

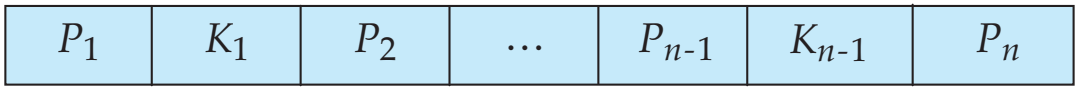
- B^+-Tree Basic Concepts
- B^+-Tree Lookup Algorithm
- B^+-Tree Insertion Algorithm
- B^+-Tree Deletion Algorithm

B+-Tree Index

- Disadvantage of indexed-sequential files
  - Performance degrades as file grows, since many overflow blocks (new blocks for index) get created.
  - Periodic reorganisation of entire file is required.
- Advantage of B+tree index files
  - Automatically reorganises itself with small, local changes, in face of insertions and deletions.
    - Reorganisation of entire file is not required to maintain performance.
- Disadvantage of B+-trees (minor)
  - Extra insertion and deletion overhead, space overhead.
- Advantages of B+trees outweigh disadvantages
  - B+-trees are used extensively, e.g. MySQL (InnoDB), PostgreSQL and SQL Server

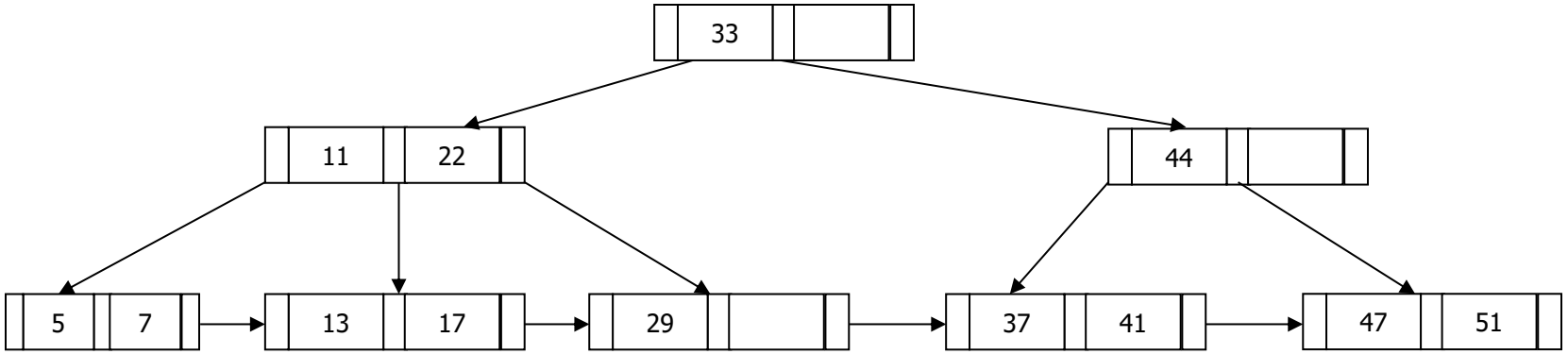
B+-Tree Basics

- B+-Tree is "short" and "Fat"
  - Disk-based: usually one node per block; large fan-out
  - Balanced (more or less): good performance guarantee
- In B+-Tree:
  - n or N (order) is number of pointers in one node
  - In typical node:
    - Pointers: $P_1, P_2, \ldots P_n$
    - Search keys (NO repetitive values): $K_1 < K_2 < K_3 < \ldots < K_{n-1}$
  - All paths (from root to leaf) have same length
  - Special cases:
    - If root is leaf, it can have between 0 and *n*-1 children
    - If root is not leaf, it must have at least 2 children
  - Each non-leaf node (inner node) must contain at least $\lceil n/2 \rceil$ pointers
  - Each leaf node must contain at least $\lceil (n-1)/2 \rceil$ keys



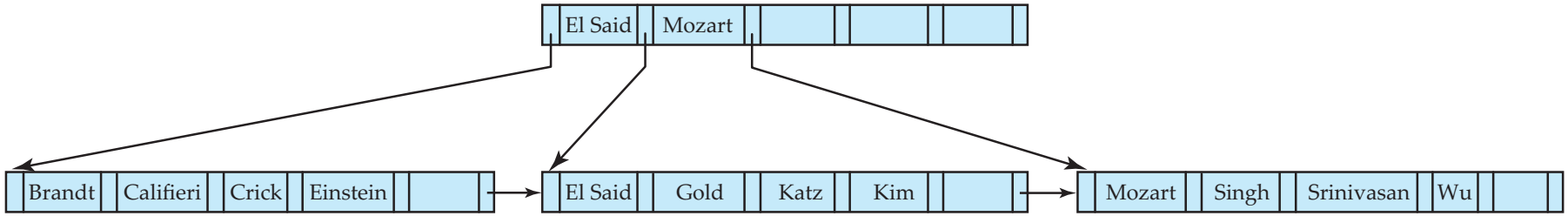
B+Tree Example

- Example B+-Tree with order n = 3
  - All paths have same length.
  - Root has (at least) two children
  - In each non-leaf node, at least $\lceil 3/2 \rceil = 2$ pointers are used
  - Each leaf node contains at least $\lceil (3-1)/2 \rceil = 1$ key



B+Tree Example – cont'd

- Another B⁺-tree example with $n = 6$
  - Root must have at least 2 children.
  - Leaf nodes must have between 3 and 5 search key values, ($\lceil (6-1)/2 \rceil$ and $6-1$).
  - Non-leaf nodes other than root must have between 3 and 6 children/pointers ($\lceil 6/2 \rceil$ and 6).

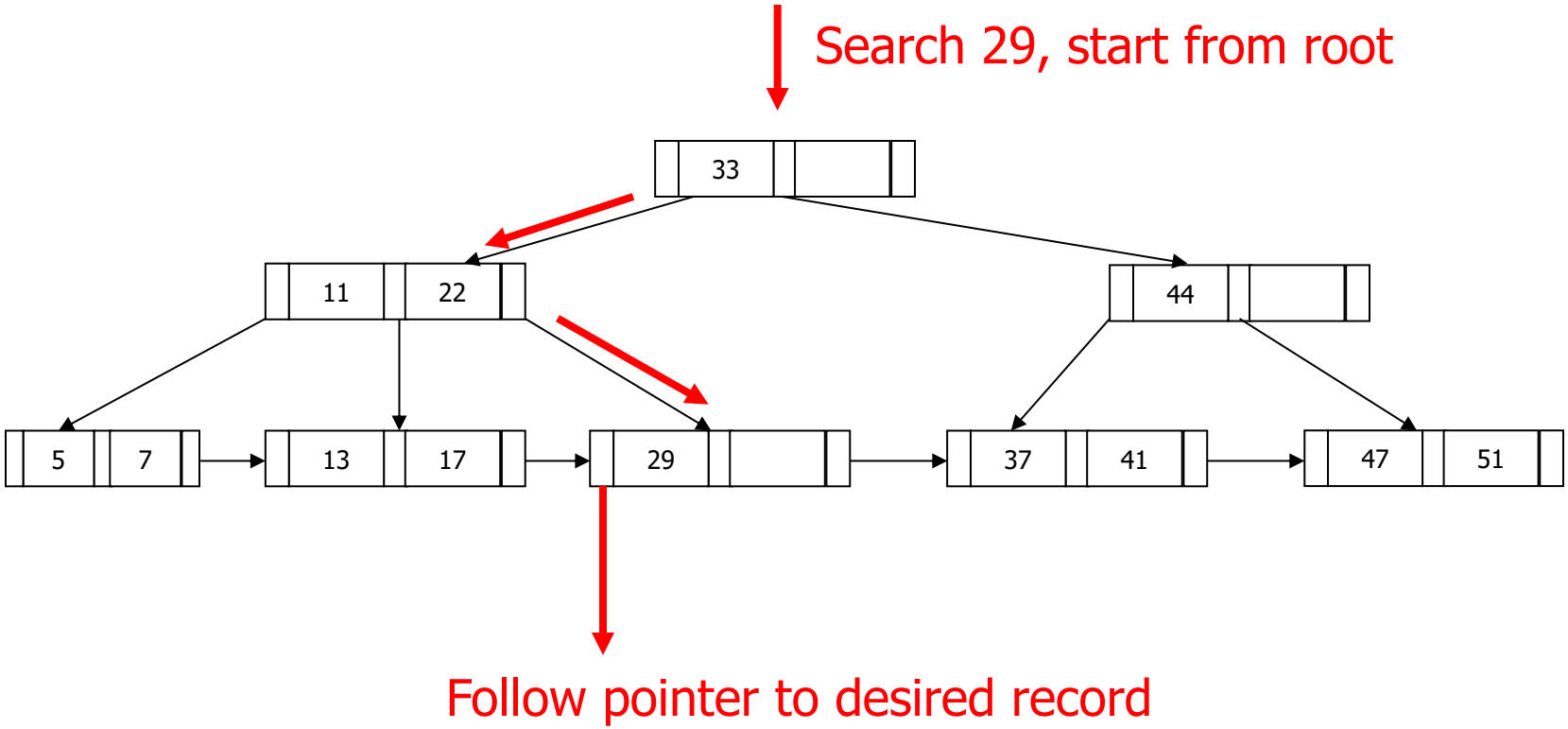


Query Algorithm on B^+-Trees

- To find record with search-key value *V*.
  - 1. Start from root: *C* = root
  - 2. While *C* is not leaf node
    - 2.1. Let i be least value such that $V \leq K_i$.
    - 2.2. If no such i exists, set *C* = last non-null pointer in *C*
    - 2.3. Else { if ($V = K_i$) Set $C = P_{i+1;}$ else set $C = P_i$}
  - 3. Let i be least value such that $K_i = V$
  - 4. If there is such value i, follow pointer P_i to desired record.
  - 5. Else no record with search-key value V exists.

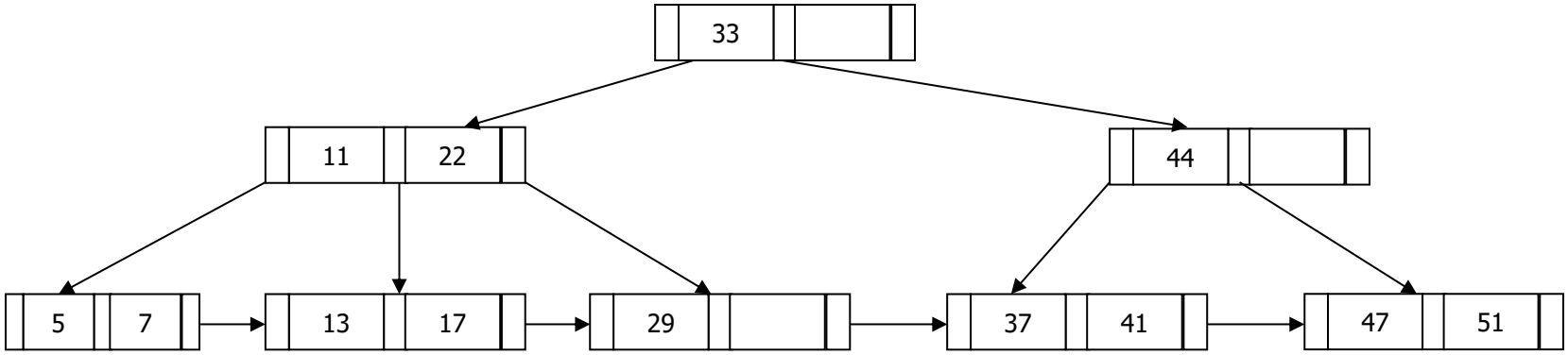
Query Example

- To find search key '29'



Query Example – cont'd

- How to find search key '31'?
- How to find search key '33'?



Observations about B+-trees

- Since inter-node connections are done by pointers, "logically" close blocks need not be "physically" close.
- Non-leaf levels of B+-tree form hierarchy of sparse indices.
- If there are K search-key values in file, height of B+-tree is *no more than* $\lceil \log_{\lceil n/2 \rceil}(K) \rceil$.
  - Level below root has at least 2 values
  - Next level has at least $2 * \lceil n/2 \rceil$ values
  - Next next level has at least $2 * \lceil n/2 \rceil * \lceil n/2 \rceil$ values
  - .. etc.

Observations about B+-trees cont'd

- Searching can be conducted efficiently.
  - Node is generally same size as block, typically 4 kilobytes
  - N is typically around 100 (40 bytes per index entry).
  - With 1 million search key values and n = 100
  - At most $\log_{50}(1{,}000{,}000) = 4$ nodes are accessed in lookup.
- Insertion and deletion to main file can also be handled efficiently, as index can be restructured in logarithmic time.

Updates on B+-Trees: Insertion

- 1. Find leaf node in which search-key value would appear
- 2. If search-key value is already present in leaf node
  - 2.1. Add record to file
  - 2.2. If necessary add pointer to bucket.
- 3. If search-key value is not present, then
  - 3.1. Add record to main file (and create bucket if necessary)
  - 3.2. If there is room in leaf node, insert (key-value, pointer) pair in leaf node
  - 3.3. Otherwise, **split** node (along with new (key-value, pointer) entry) as discussed in next slides.
- 4. Splitting of nodes proceeds upwards till node that is not full is found.
  - In worst case root node may be split, increasing height of tree by 1.

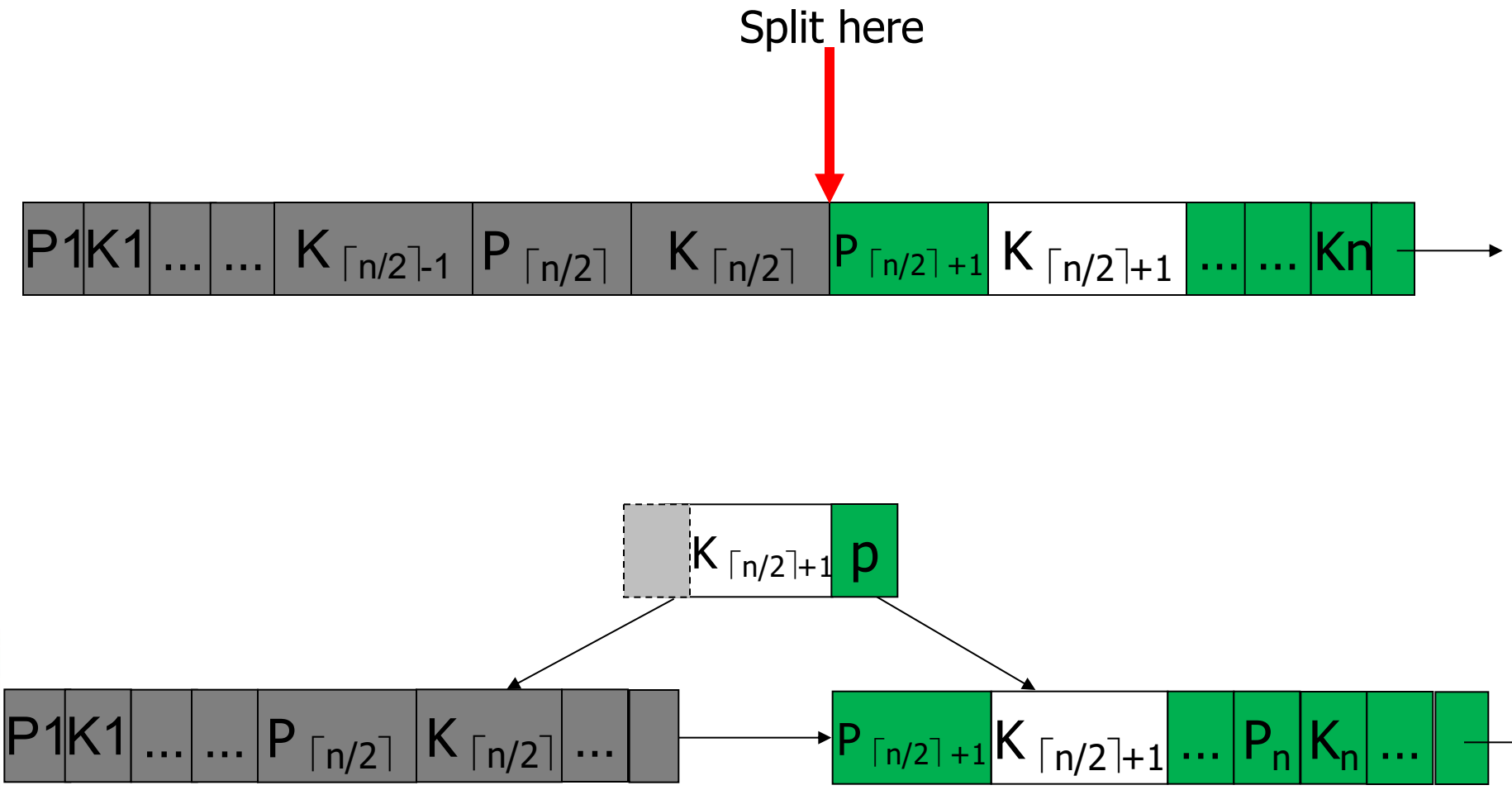
Updates on B+-Trees: Insertion cont'd

- Splitting leaf node:
  - Take original node and search key being inserted in memory area M in sorted order. There are n search key values in total.
  - Place first ⌈n/2⌉ in original node, and rest in new node.
  - Let new node be p, and let k be least key value in p. Insert (k, p) into parent of node being split.
  - If parent is full, split it and propagate split further up.

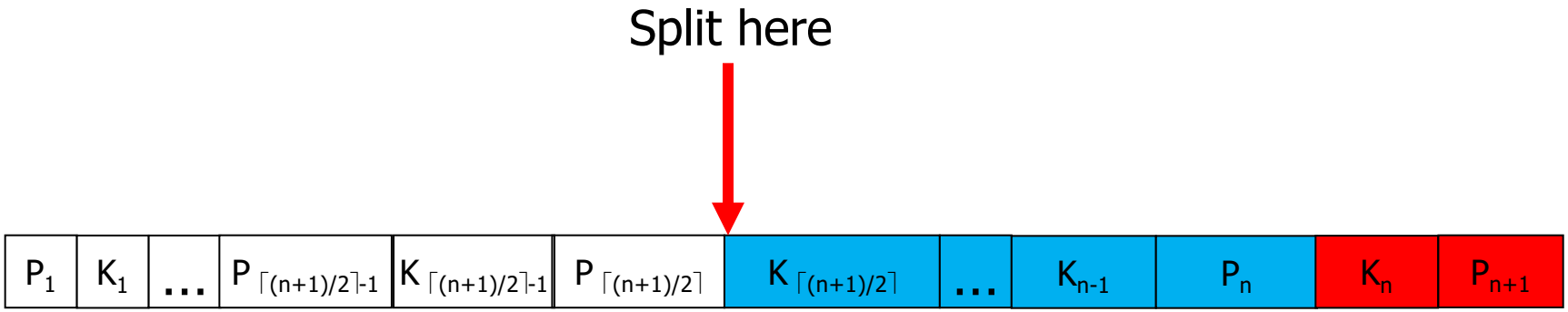
Updates on B$^+$-Trees: Insertion cont'd

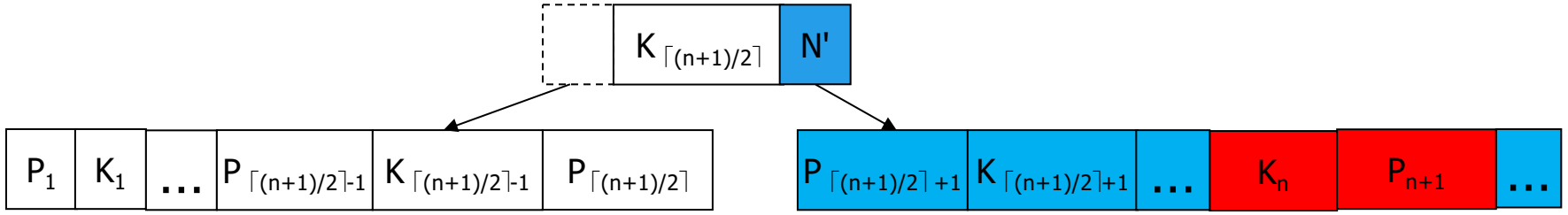
- Splitting non-leaf node: when inserting (k, p) into already full internal node N
  - Copy N to in-memory area M with space for n+1 pointers and n keys
  - Insert (k, p) into M in sorted order
  - Copy $P_1, K_1, \ldots, K_{\lceil (n+1)/2 \rceil - 1}, P_{\lceil (n+1)/2 \rceil}$ from M back into node N
  - Copy $P_{\lceil (n+1)/2 \rceil + 1}, K_{\lceil (n+1)/2 \rceil + 1}, \ldots, K_n, P_{n+1}$ from M into newly allocated node N'
  - Insert $(K_{\lceil (n+1)/2 \rceil}, N')$ into parent of N

Splitting Leaf Node

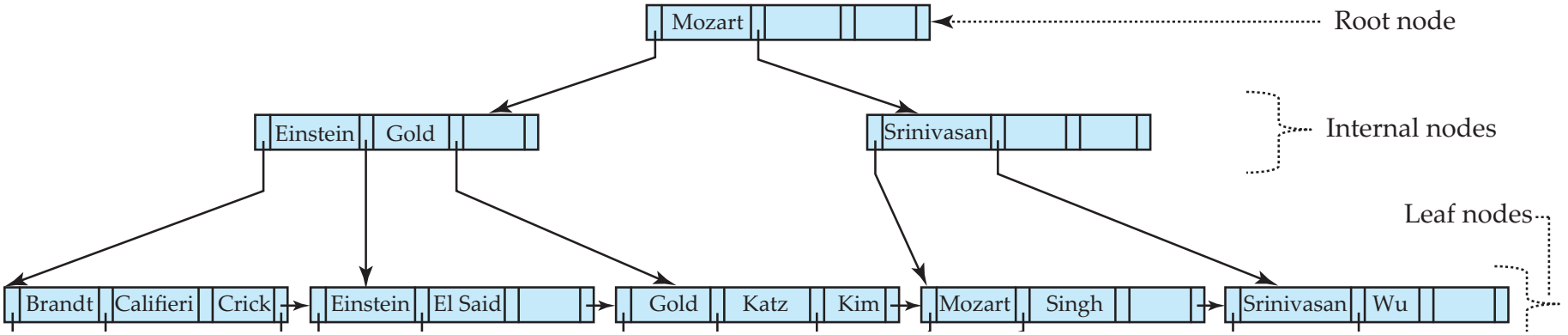


Splitting Non-leaf Node



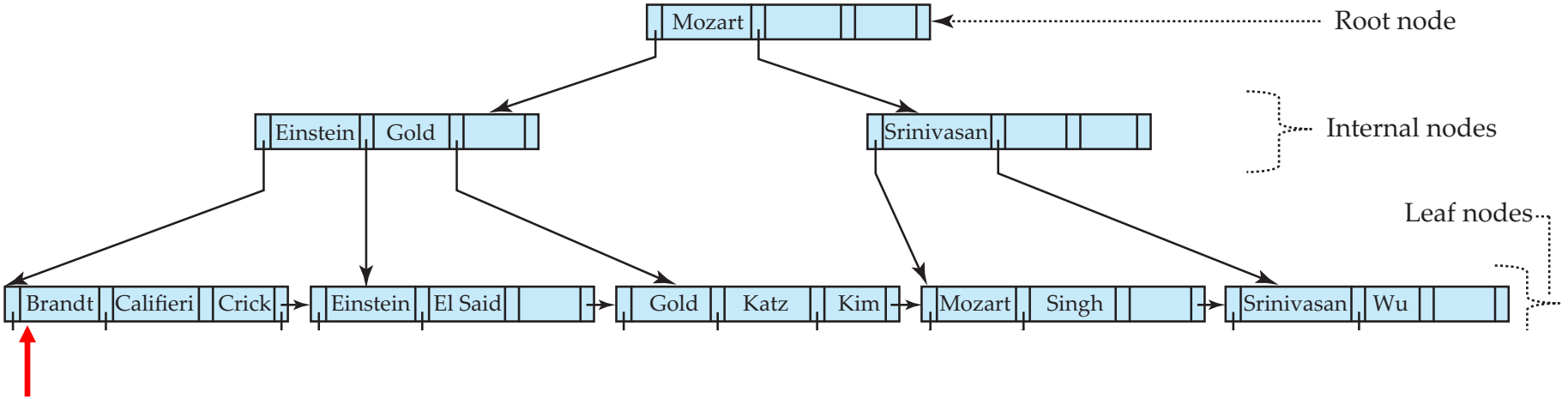


Insertion Example

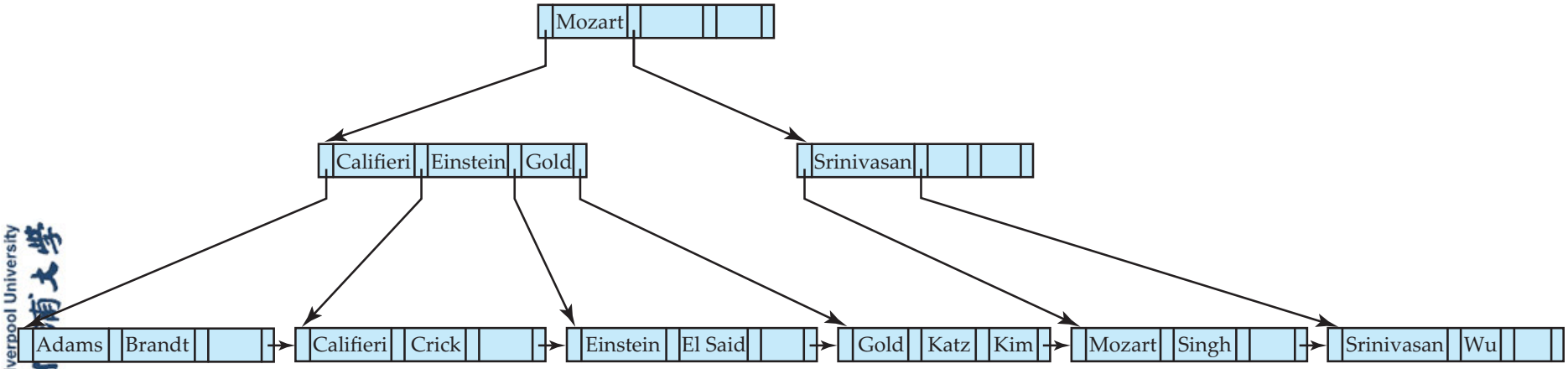


Insert key “Adams” into initial B+-Tree above

Insertion Example – cont'd



Insert "Adams" here



B+-Tree before and after insertion of "Adams"

Updates on B$^+$-Trees: Deletion

- 1. Find record to be deleted, and remove it from bucket (if present); remove (search-key value, pointer) from leaf node if there is no bucket or if bucket has become empty.
- 2. If node has too few entries due to removal (underfull):
  - And if entries in node and its sibling fit into single node, then merge siblings.
  - Otherwise, if entries in node and its sibling do not fit into single node, then redistribute pointers.
  - Both merge siblings and redistribute pointers need to be considered at both leaf and non-leaf levels.
  - Node deletions may cascade upwards till node which has $\lceil n/2 \rceil$ or more pointers is found.
- 3. If root node has only one pointer after deletion, it is deleted and sole child becomes new root (height decreased by 1).

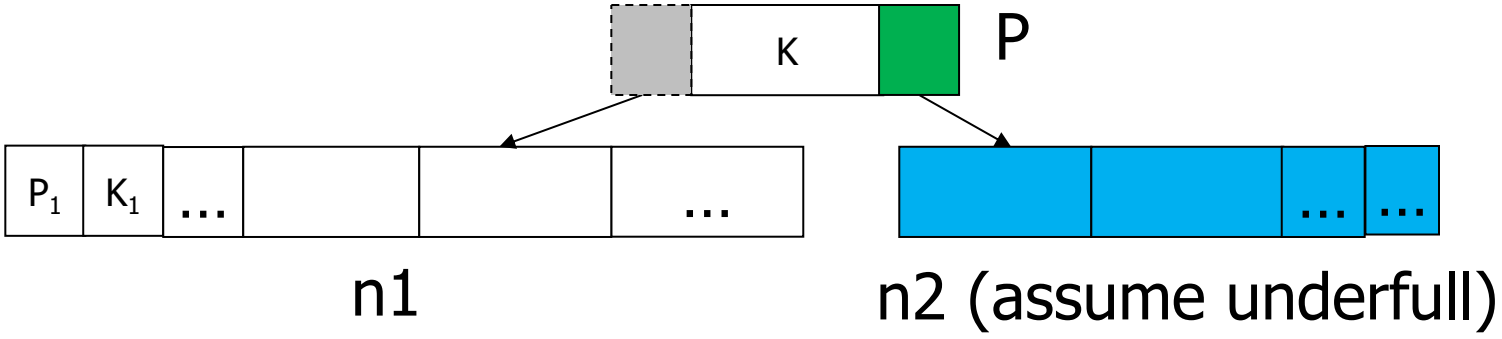
Perform merge siblings first and then redistribute pointers

Note on Insertion and Deletion

- There is difference between *inserting/ deleting search key from B+ tree* and *inserting/* deleting record from file.
- Take deletion as example
  - Deleting search key from B+-tree
    - Key is to be deleted from tree
    - All records indexed by that key should have all been deleted already
  - Deleting record from file
    - Only record is to be deleted from file
    - Key indexing that record might still present in B+-tree.

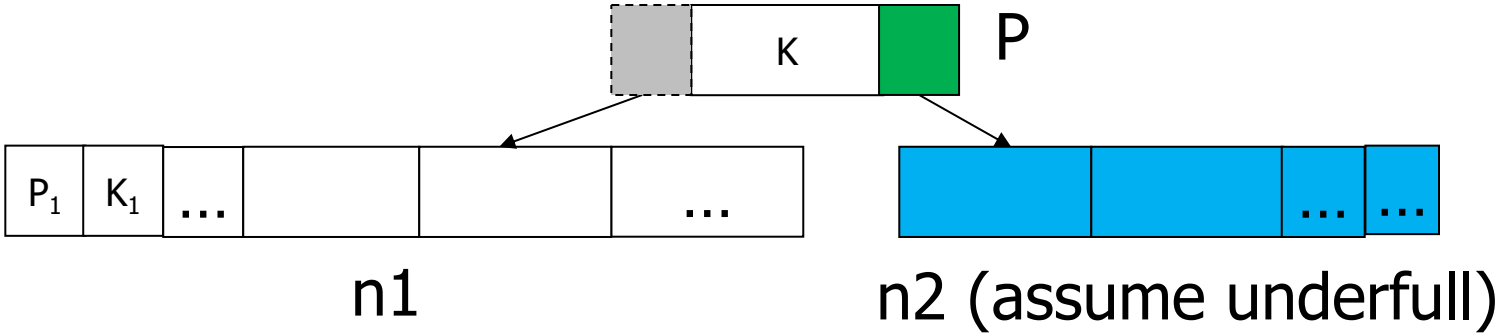
Merge Siblings – at Leaf Node

- Merge siblings n1 and n2
- Delete K (and appropriate pointer) from parent P
- Arrange pointers



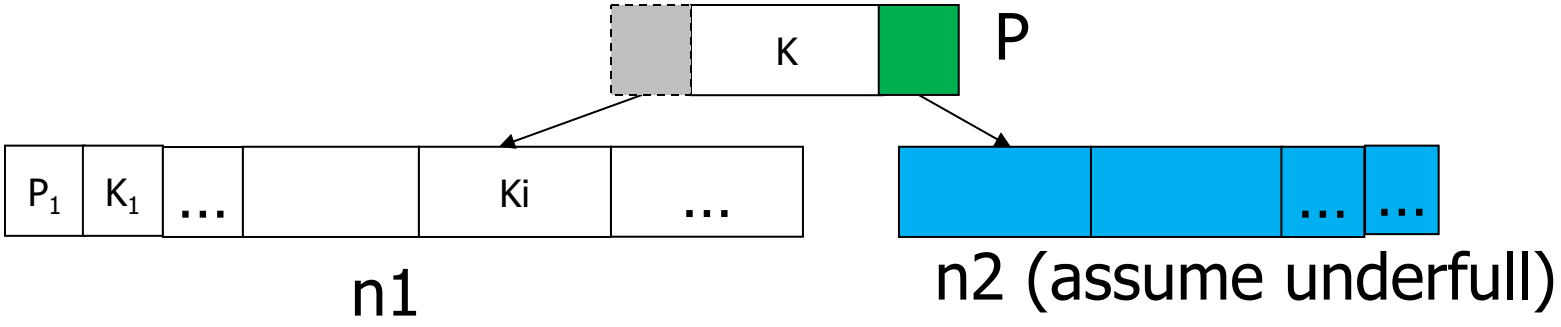
Merge Siblings – at non-Leaf Node

- Merge siblings n1 and n2, and K
- Delete K (and the appropriate pointer) from parent P
- Arrange pointers



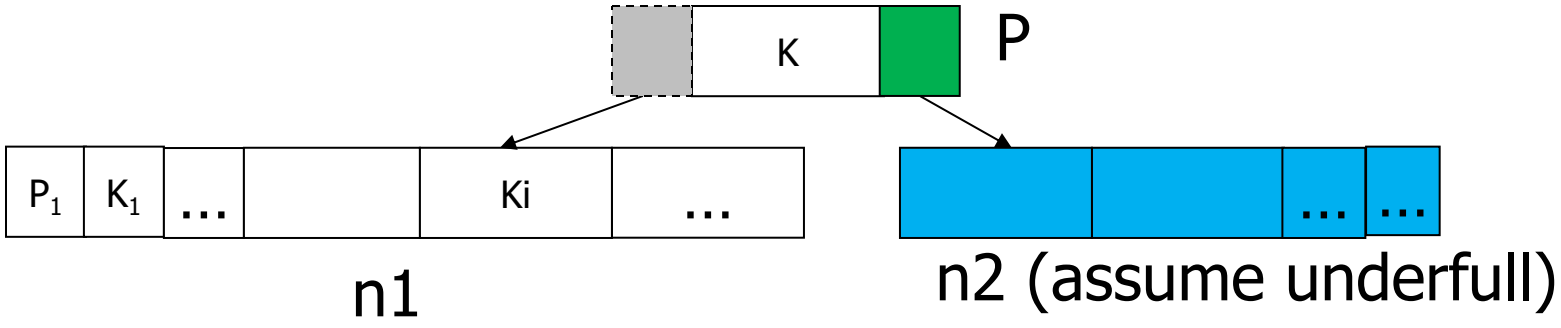
Redistribute Pointers – at Leaf Node

- Copy Ki from n1 and add it to n2
- Delete Ki from n1
- Replace old value K in parent P with Ki
- Arrange pointers



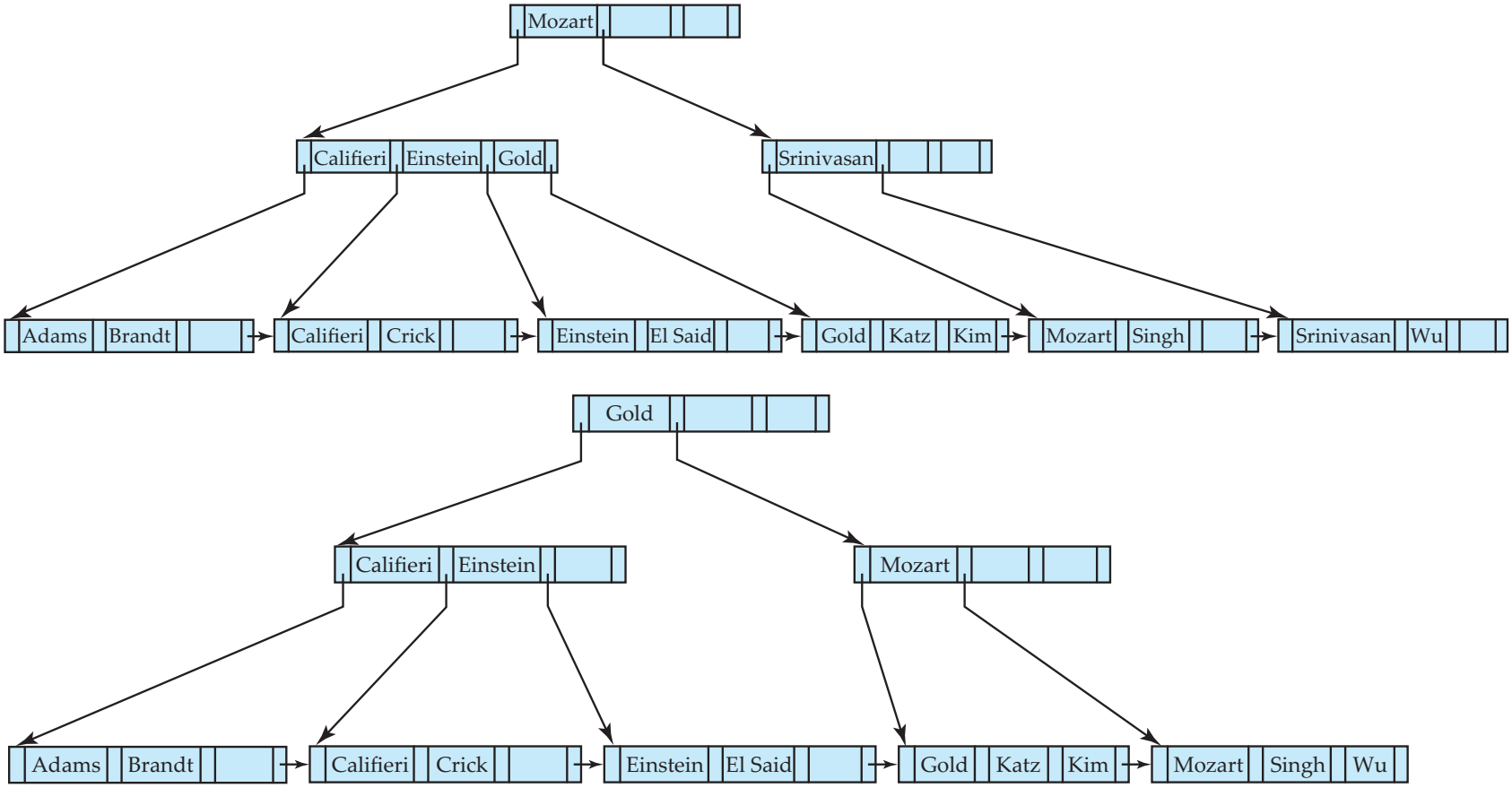
Redistribute Pointers – at non-Leaf Node

- Copy K from parent P and add it to n2
- Replace old value K in parent P with Ki from n1
- Delete Ki from n1
- Arrange pointers



Deletion Example

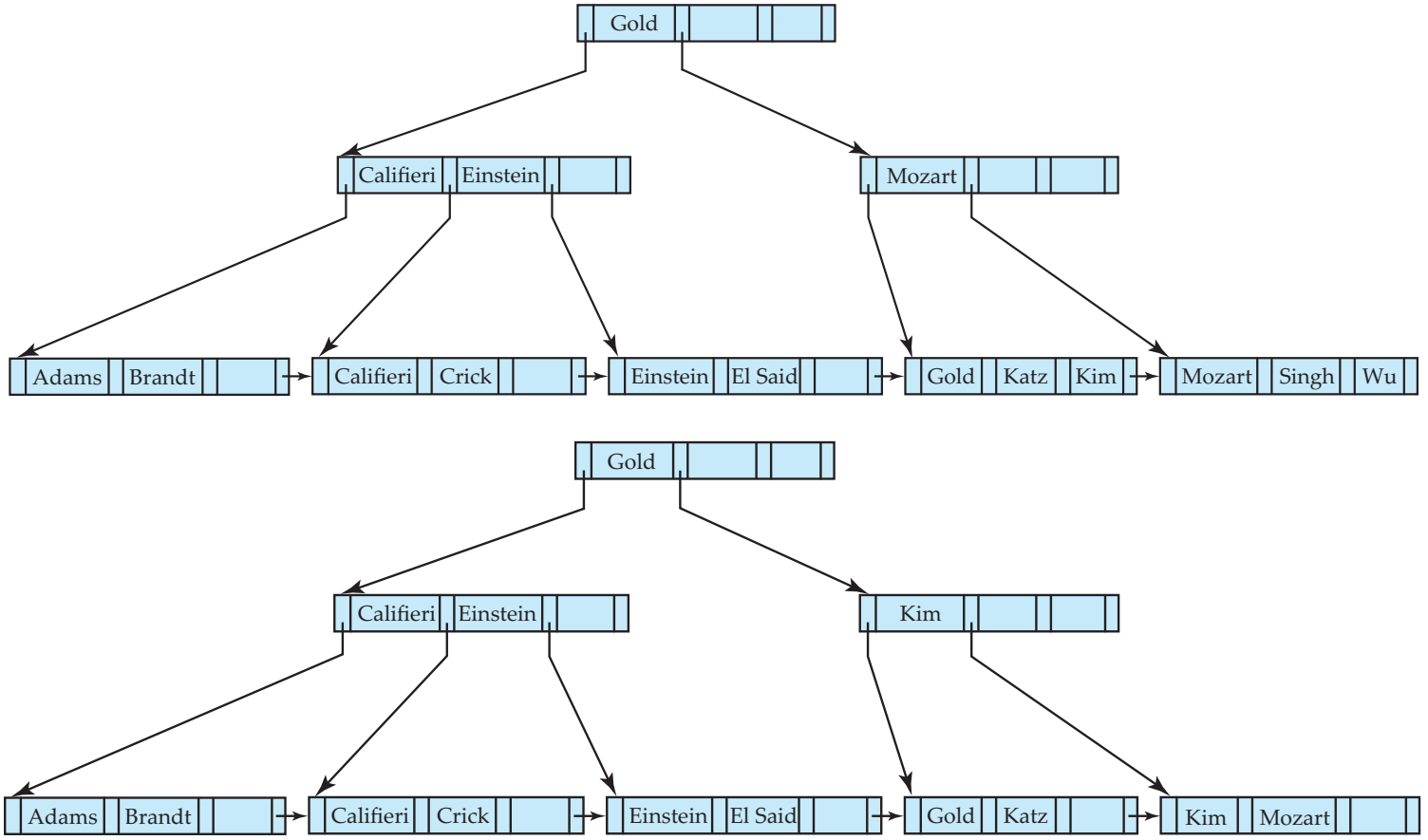
Before and after deleting “Srinivasan”



- Deleting “Srinivasan” causes merging of under-full leaves
- And then **redistributing pointers at non-leaf**

Deletion Example cont'd

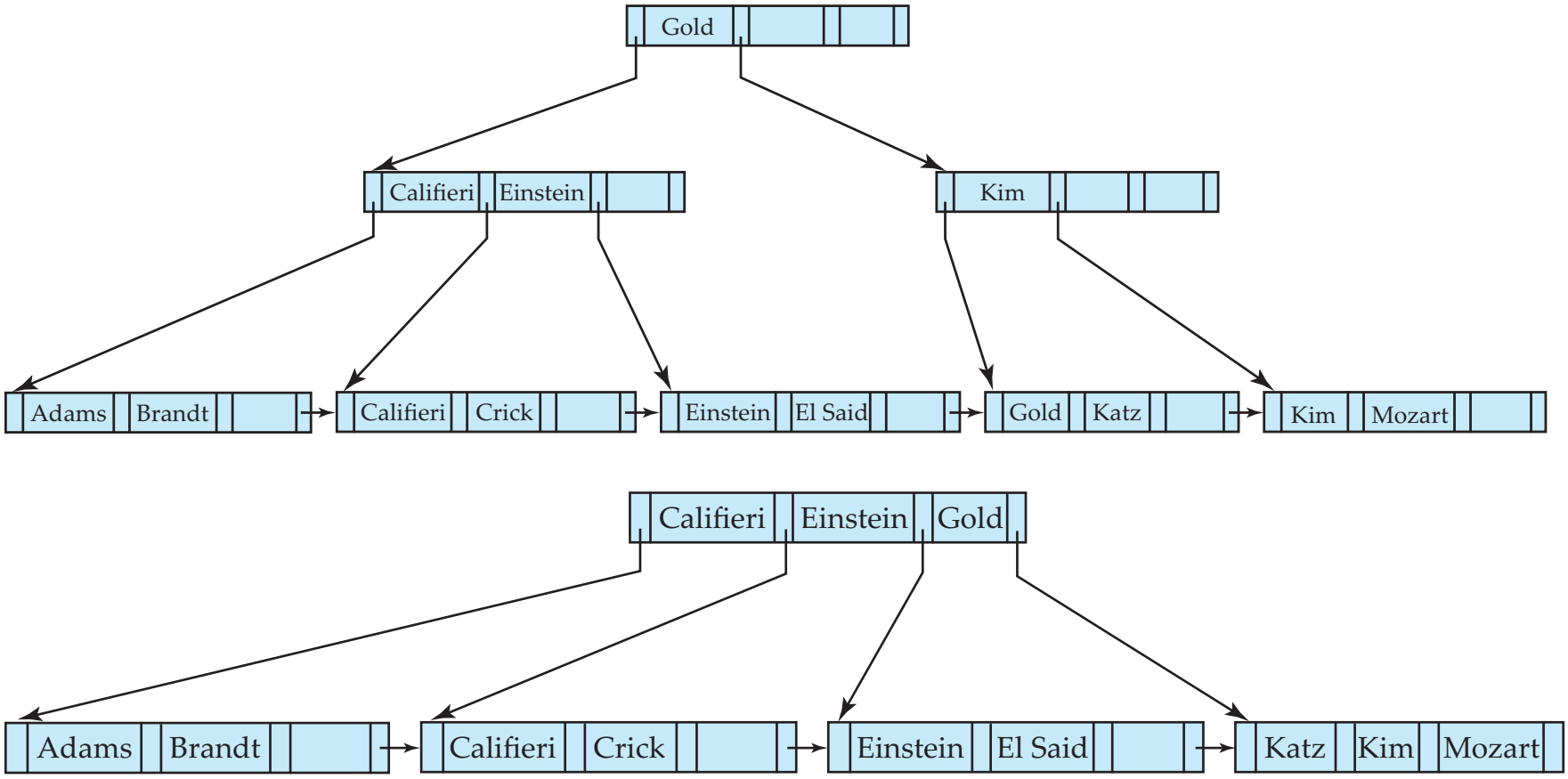
Before and after deleting "Singh and Wu"



- Deleting "Singh" can be done directly
- Deleting "Wu": **redistributing pointers at leaf level**

Deletion Example cont'd

Before and after deleting "Gold"



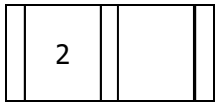
- Deleting "Gold": **merge leaf nodes first and then merge non-leaf nodes**

B+ Tree Construction Example

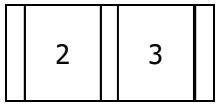
- Initial B⁺-tree with n=3 is empty. Construct B⁺-tree by inserting following key values in exact order.
  - (2, 3, 5, 7, 11, 13, 17)

B+ Tree Construction Example – cont'd

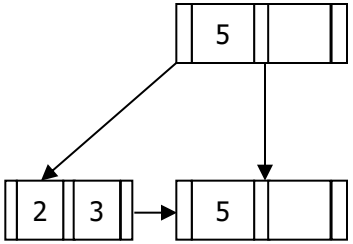
1. Insert 2



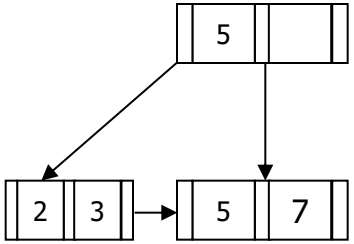
2. Insert 3



3. Insert 5

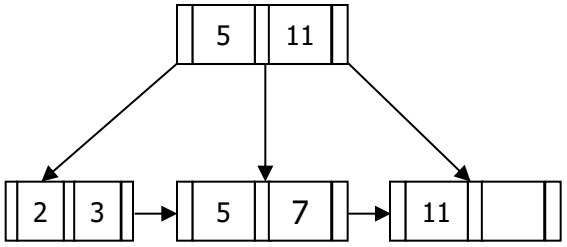


4. Insert 7

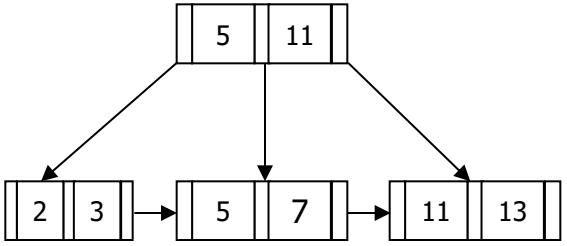


B+ Tree Construction Example – cont'd

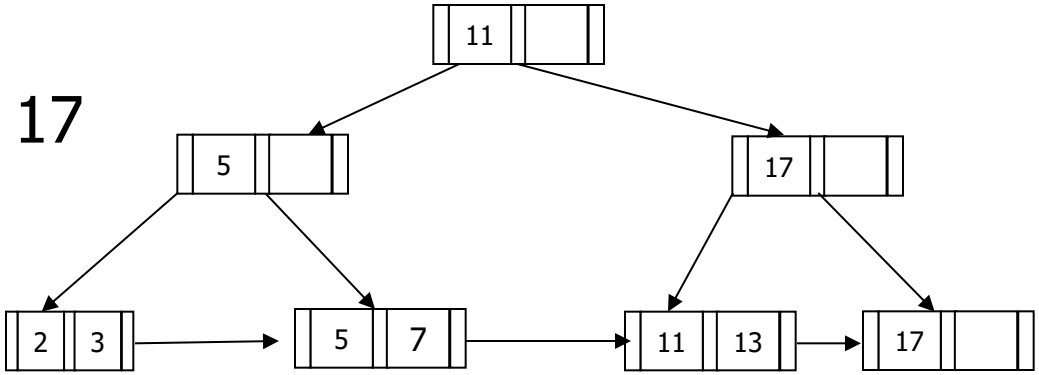
5. Insert 11



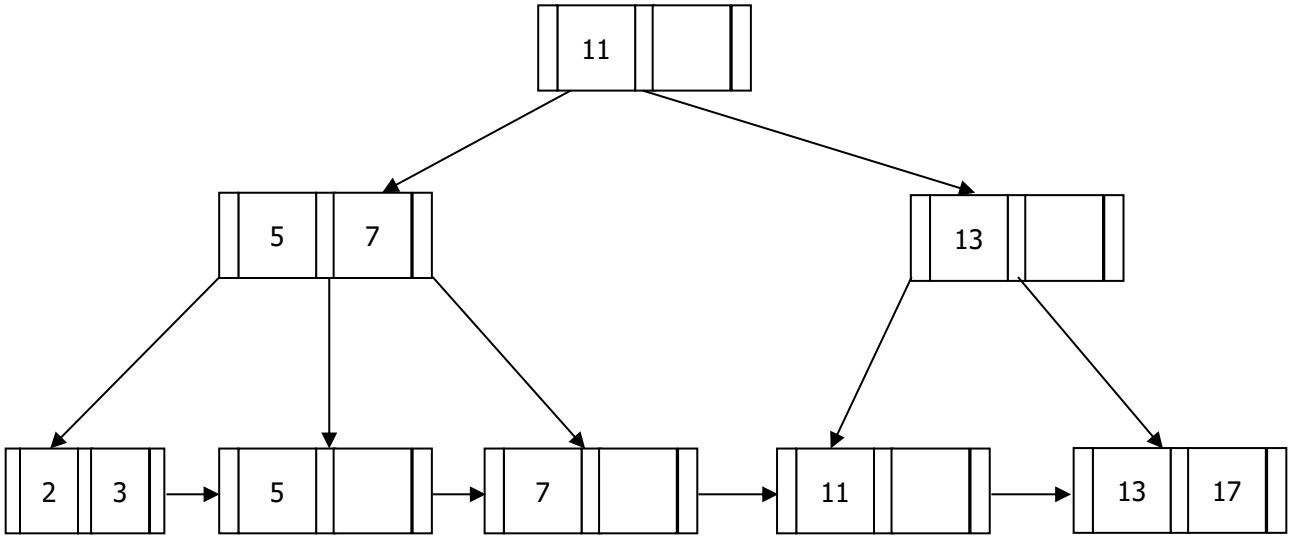
6. Insert 13



7. Insert 17



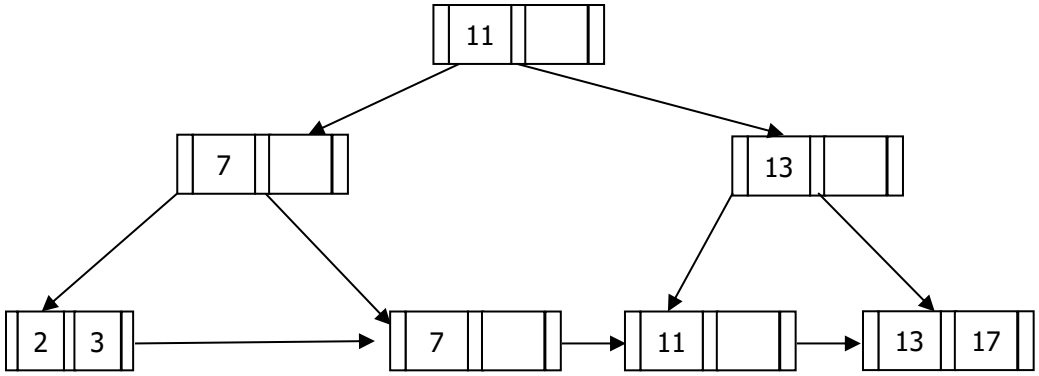
Another Deletion Example



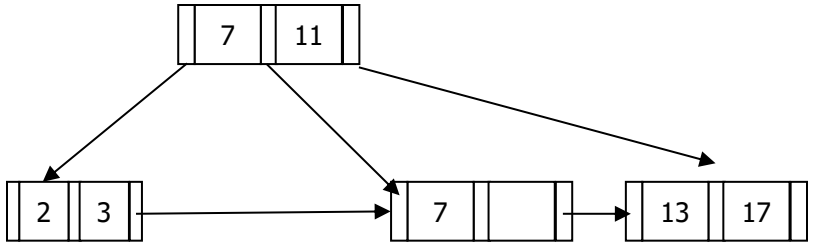
- Initial tree shown above
- (1) Delete 5, then delete 11, what does tree look like?
- (2) With initial tree, delete 11, what does tree look like?

Another Deletion Example – cont'd

- (1) Delete 5: merge with either left or right sibling.

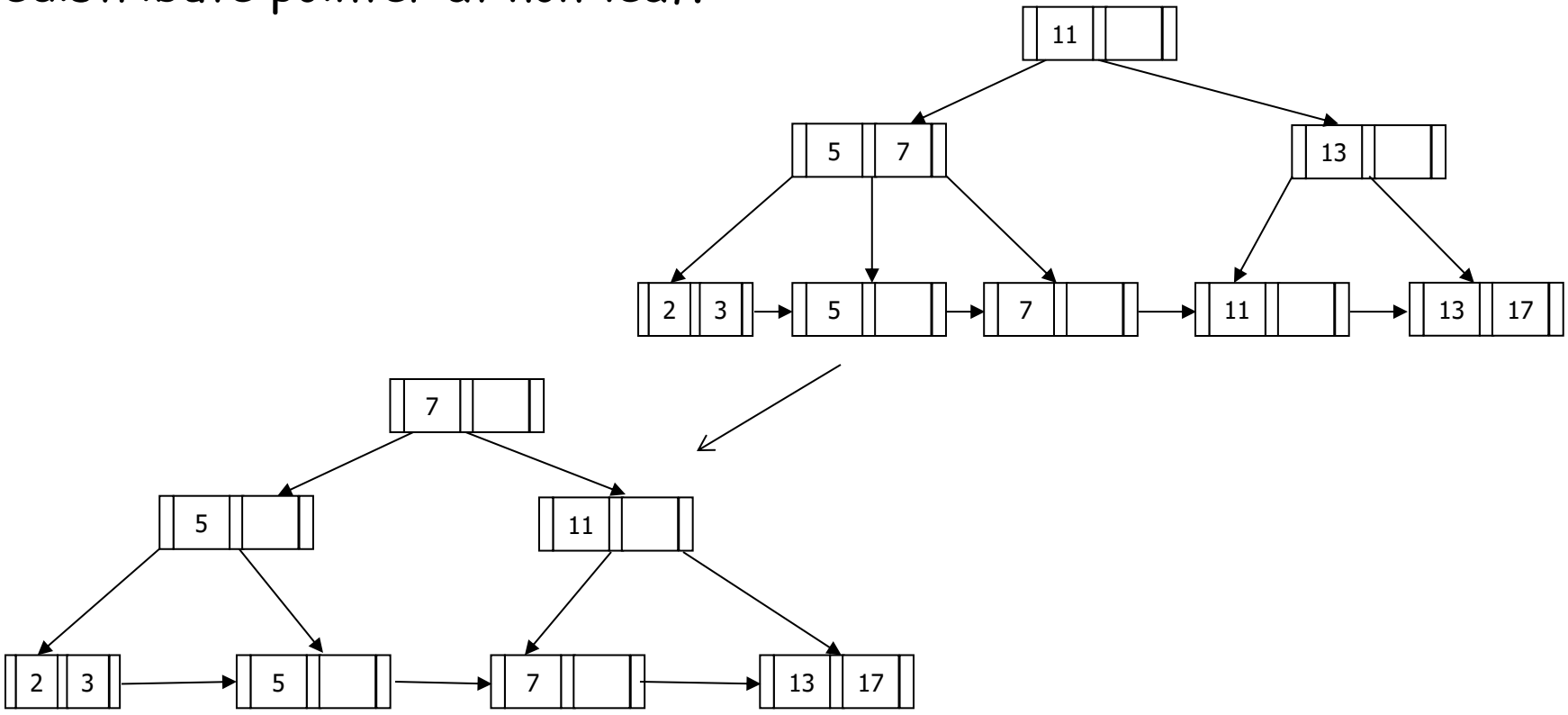


- Then delete 11: merge sibling at leaf and then merge again at non-leaf.



Another Deletion Example – cont'd

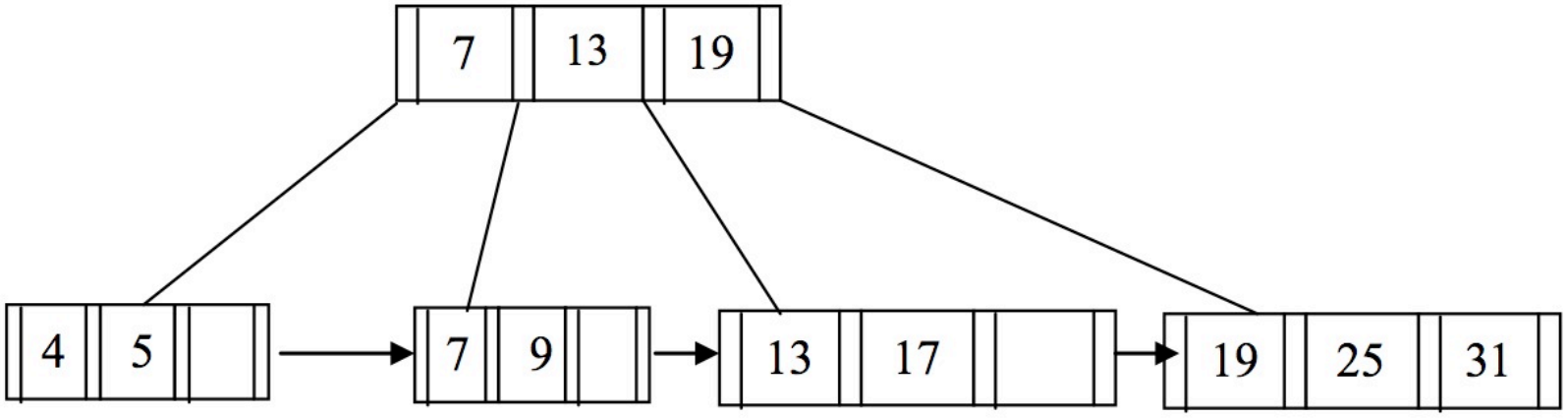
- (2) With Initial tree, Delete 11: merge sibling at leaf and then redistribute pointer at non-leaf.



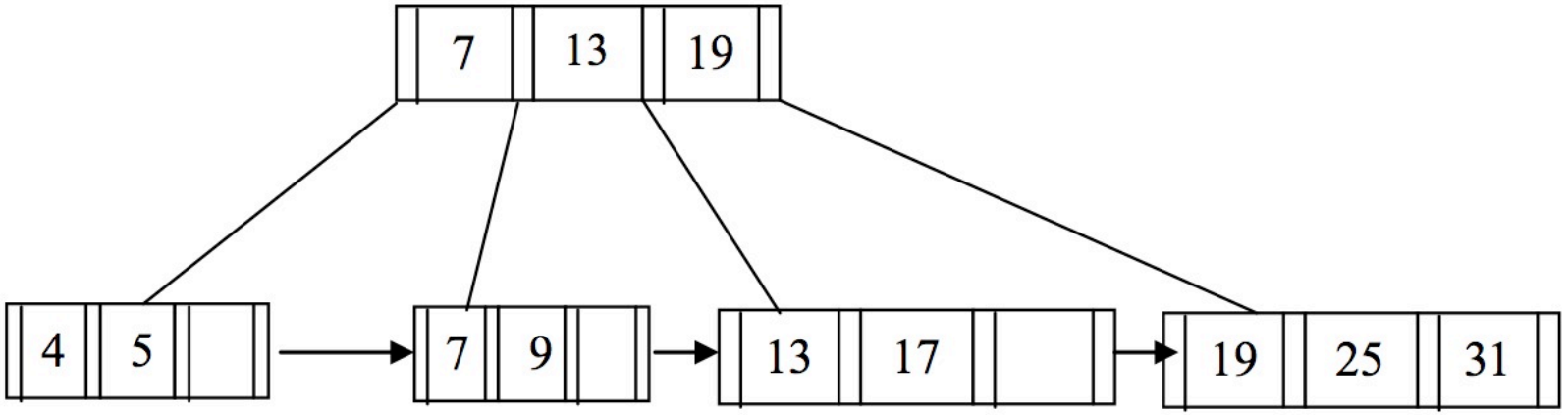
Note: redistribute pointer at leaf also solves problem. However, as algorithm in textbook suggests, perform merge sibling first.

Insertion and Deletion Example

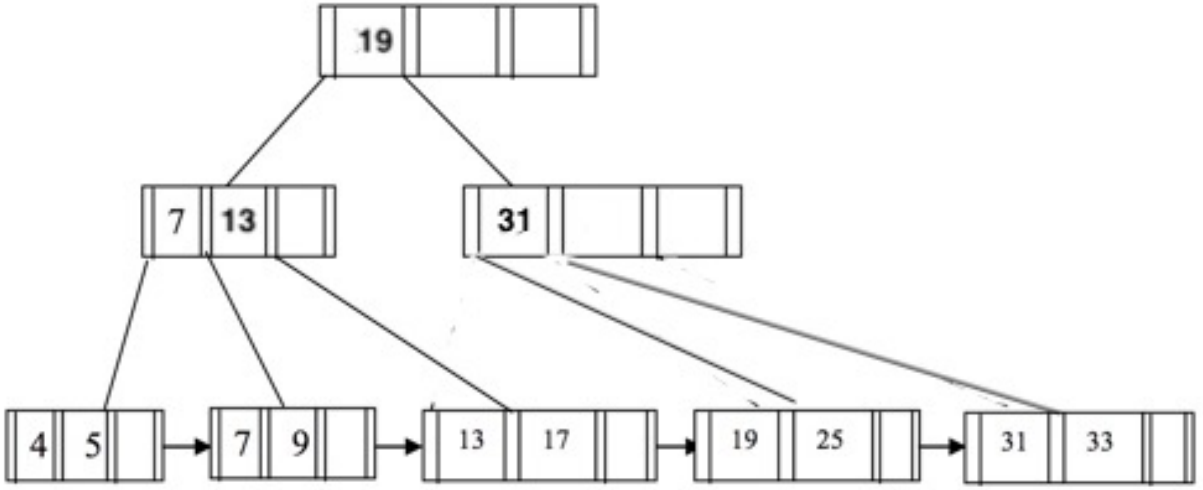
- Consider following B+-tree with n=4.
- Draw trees after each of following update operations is performed (Subsequent operations are performed based on previous ones).
  - 1. Insert 33
  - 2. Insert 11
  - 3. Delete 25



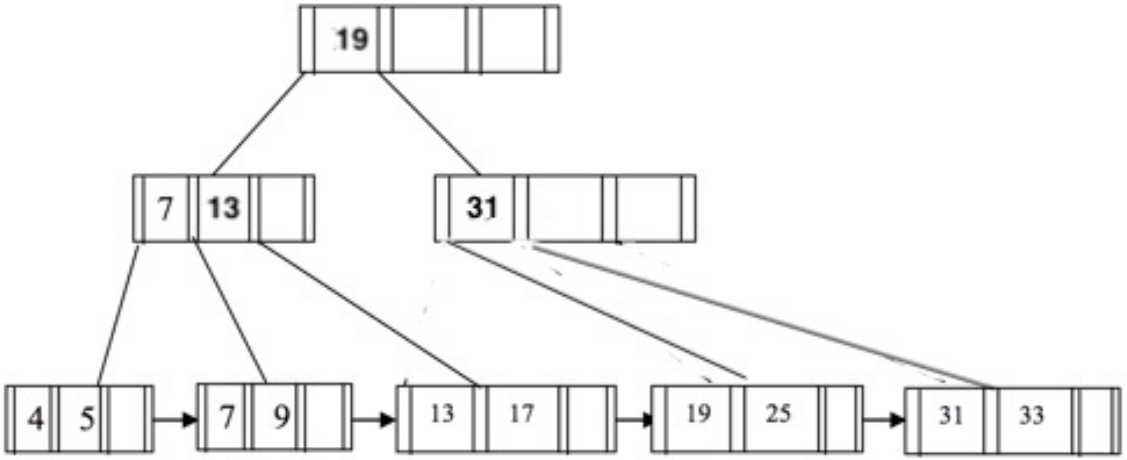
Insertion and Deletion Example – cont'd



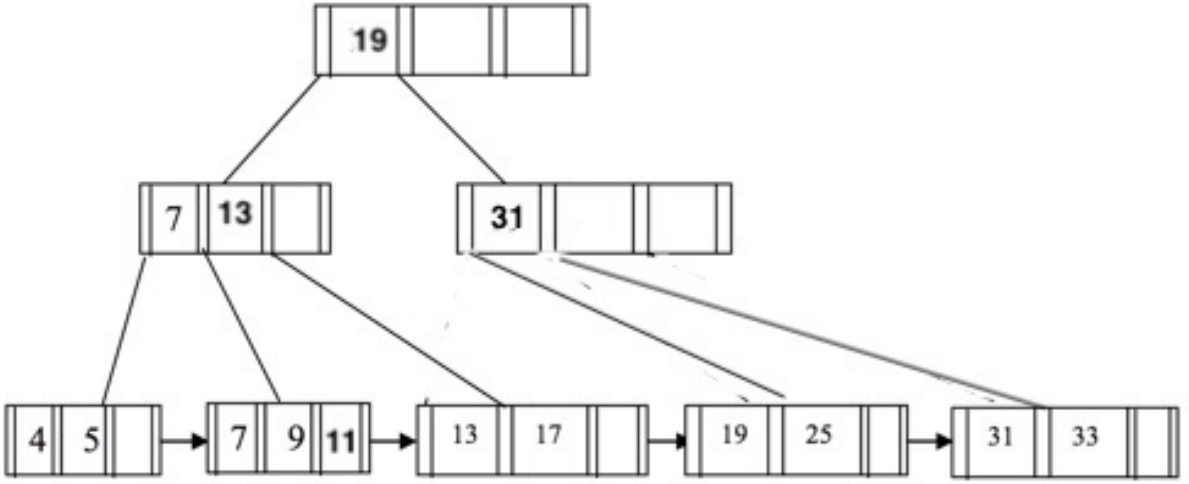
- Insert 33 (split at leaf and then split at non-leaf)



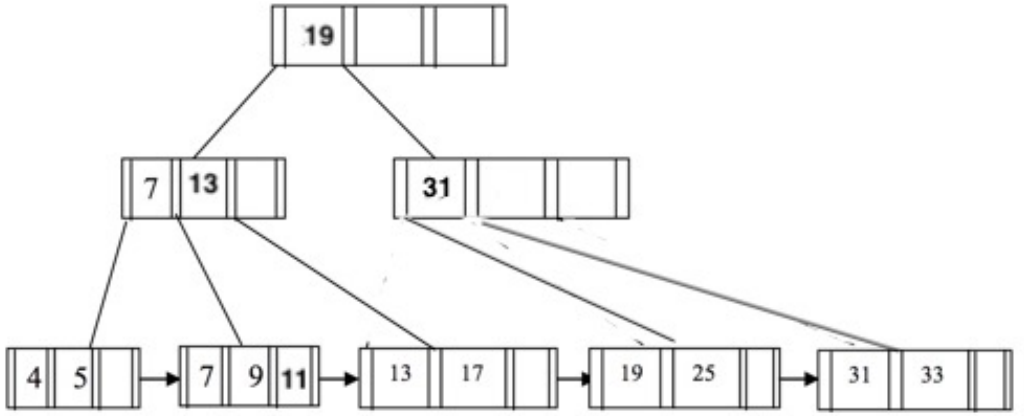
Insertion and Deletion Example – cont'd



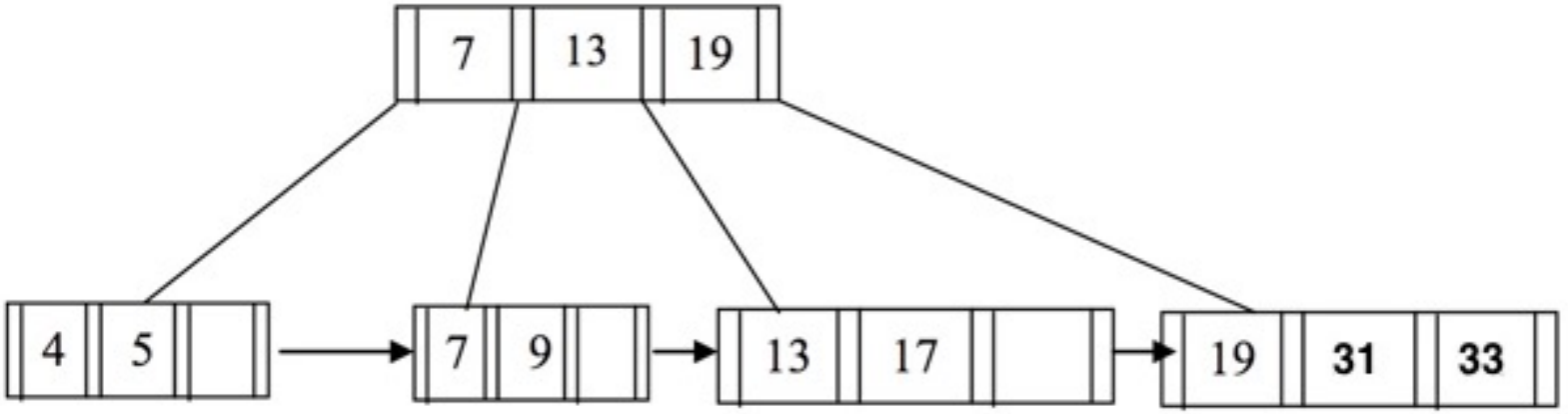
- Insert 11



Insertion and Deletion Example – cont'd



- Delete 25 (merge siblings at leaf and then merge again at non-leaf)



End of Lecture

- Reading
  - Database System Concepts, 7th edition, chapter 14.1, 14.2, 14.3